Literature Review: Reliable Transport and Congestion Control over UDP

Oct 9, 2026 (updated to cover Smart-RUDP)

Prepared for Computer Networks by Harshavardhan Mallela (24BAI0274), Rohit Arya (24BCE0819) and Shloak Sinha (24BDS0378). Faculty: Anuradha G. This review traces each mechanism in our RUDP protocol and its Smart-RUDP extension back to its source, names the application area and the key researcher, and lists nine references.

Application area (case study)

RUDP belongs to user-space reliable transport over UDP, the architecture QUIC uses to deliver reliable, congestion-controlled traffic for HTTPS [5][6]. Our protocol is a teaching and testing vehicle, not a production system. It shows how a handshake, sequence numbers, retransmission, a sliding window and congestion control fit together in a small C program, and how a learned window policy can replace the fixed rule.

Three settings where this matters:

- **Evolving transports:** building on UDP lets a transport change without waiting for operating systems or network operators to update [5].
- **Education and research:** a protocol you can read end to end makes congestion-control behaviour visible, as the window sawtooth in our results shows.
- **Loss-resilience testing:** emulated loss, Wireshark captures and benchmarks show how a transport reacts when the network misbehaves.

Background and problem

A reliable transport needs more than retransmission: it must also slow down when the network is congested. UDP provides neither. It delivers datagrams with no guarantee of arrival, order or pacing, while TCP adds all three.

The cost of getting this wrong appeared in October 1986, when the Internet suffered its first congestion collapse and throughput between Lawrence Berkeley Laboratory and UC Berkeley fell from 32 Kbps to 40 bps [1]. Jacobson and Karels traced much of the cause to how transport protocols were implemented, not to the protocols themselves [1]. A sender that blindly retransmits makes congestion worse, which is why RUDP pairs its retransmission logic with a congestion window.

Foundations: slow start, AIMD and fast retransmit

Three works define the mechanisms RUDP reuses.

1. Jacobson and Karels (1988) [1] introduced the packet-conservation principle and added new algorithms to the 4BSD TCP, among them round-trip-time variance estimation, exponential retransmit backoff, slow start and congestion avoidance.
2. Chiu and Jain (1989) [2] analysed increase/decrease rules and showed that additive increase with multiplicative decrease converges to an efficient and fair state regardless of the starting point. That result is why RUDP grows its window linearly and cuts it by half.
3. RFC 5681 (2009) [3] standardises four algorithms: slow start, congestion avoidance, fast retransmit and fast recovery, drawing on Jacobson's work.

Beyond Reno: CUBIC

CUBIC replaces Reno's linear window growth with a cubic function of the time since the last congestion event, which scales better on fast, long-distance paths [4]. It replaced its predecessor BIC-TCP as the default algorithm in Linux [4]. We did not implement CUBIC. It remains a planned comparison.

Learning-based congestion control (the AI part of this project)

Rule-based schemes such as AIMD and CUBIC use fixed formulas. AIMD halves the window on every loss signal, so it cannot tell a random wireless loss from congestion, and it grows by one packet per round trip once past the threshold, which is slow on a long satellite path. Recent work applies reinforcement learning to this problem.

- **Midhula and Arun Raj Kumar (2024) [8]** present an adaptive congestion control protocol for wireless networks that uses deep reinforcement learning, motivated by the weakness of fixed rules on wireless links.
- **Cao et al. (2023), DRLFcc [9]** use deep reinforcement learning to drive congestion control during TCP fast recovery in high-loss wireless networks.

Our Smart-RUDP follows this idea at a smaller scale. It replaces the window-update rule with a tabular Q-learning agent (80 states built from the RTT ratio, loss fraction and window size; 5 actions that scale the window by 0.5, 0.85, 1.0, 1.15 or 1.5; a reward that favours throughput and penalises delay growth and retransmissions). Both papers use deep neural networks. Our C agent uses a lookup table, which is simpler and inspectable; the chat also has a small deep Q-network (two hidden layers of 32 units) trained in simulation. In our experiments on emulated 5G-like, lossy-wireless and satellite-like paths, Smart-RUDP matched AIMD on the lossy path and was slower on the other two, with somewhat fewer retransmissions. We report this as an honest negative result, not a success.

Reliable transport on UDP in practice: QUIC

QUIC shows that reliable, congestion-controlled transport can be built on UDP at Internet scale. Langley et al. describe an encrypted, multiplexed, low-latency transport deployed on Google's servers, in Chrome and in YouTube, carrying an estimated 7% of Internet traffic at

the time of the paper [5]. Its congestion control is pluggable [5]. RFC 9000 standardises QUIC version 1: packets travel inside UDP datagrams to ease deployment on existing networks, and a credit-based scheme bounds how much data can be sent [6]. RUDP shares the UDP foundation and the idea of implementing transport logic in user space, but it has no encryption or stream multiplexing.

Testing methodology: NetEm and Wireshark

NetEm is the Linux traffic-control module that adds delay, loss, duplication and corruption to packets leaving a network interface, and it is limited by kernel timer granularity [7]. We used it to drop 20% of packets on the loopback interface during a packet capture. Wireshark served as ground truth: bursts of retransmitted data appeared about 500 ms apart, matching the protocol's 500 ms timeout, and the client logged 12 timeouts in the same run.

Scope note: the benchmark sweep from 0% to 40% loss used a loss simulator built into the client, and the Smart-RUDP comparison used an in-program path emulator, not NetEm. We did not compare against kernel TCP.

How RUDP fits the literature

Mechanism	Source	RUDP implementation	Gap
Slow start and congestion avoidance	[1][3]	cwnd grows by 1 per ACK below ssthresh (16), then by 1/cwnd per ACK	No receiver-advertised window; cwnd capped at 32
Multiplicative decrease	[2][3]	Fast retransmit: ssthresh = cwnd/2, cwnd = ssthresh. Timeout: ssthresh = cwnd/2, cwnd = 1	Simplified fast recovery
Fast retransmit	[3]	Resend after 3 duplicate ACKs	Go-Back-N resends the whole window on timeout; no SACK
Retransmission timer	[1]	500 ms timeout (Smart-RUDP client adds adaptive timeout with backoff)	Original client has no RTT estimation
Window growth function	[4]	Linear growth only	CUBIC not implemented
Learned window policy	[8][9]	Tabular Q-learning (C) and a small DQN (chat, Python)	Table did not beat AIMD in Section 8; DQN is simulated only
Reliable transport on UDP	[5][6]	Custom 13-byte header, three-way handshake, checksum	No encryption or stream multiplexing
Model-based baseline	[13]	Simplified BBR-style policy (`--mode bbr`): bandwidth and min-RTT filters, pacing-gain cycle, window = 2 x BDP	No Linux BBR, no ProbeRTT (min-RTT expiry instead), no BBRv3 loss response
Learned vs human-derived control	[10]	Neural agent compared with tuned AIMD, CUBIC and the BBR-style policy on constant, dynamic and random paths	Simulation and loopback netem only

Mechanism	Source	RUDP implementation	Gap
Fairness of learned controllers	[11]	Two neural flows with equal and unequal RTTs (Jain 0.84 at +150 ms vs 0.64 for AIMD)	Not trained for fairness; six pairs per cell
Certified learned controllers	[12]	Interval bound propagation on the 5-32-32-5 network: 43% of decisions certified at +-0.001, 0% at +-0.03	The network was not trained with certificates
Network emulation	[7]	Loss on loopback with tc/netem; in-program emulator for Smart-RUDP	Delay, jitter and reordering only in the emulator

Research gaps this project now addresses

Three recent papers were read for the gap analysis; each gap became a test (Section 8.18 to 8.22 of the report). **The authors should read these papers themselves before citing them.**

1. **Dynamic conditions** [10]. Learned controllers are usually evaluated on fixed paths. The `--dyn` option changes bandwidth and delay during a transfer; the neural agent was fastest in all four cases (bandwidth drop, rise, delay jump, hand-over pattern), by 6% to 53% against tuned AIMD, with 1% to 6% retransmissions.
2. **Strong human-derived baselines** [10][13]. A BBR-style policy was added. It beats tuned AIMD on four of five paths, and is the fastest on the 2% lossy path; the neural agent is faster on the satellite-like and fast paths. A first version failed (stall at 10% loss, 163 s after a delay jump) and was fixed.
3. **Fairness generalization** [11]. Two neural flows with unequal round-trip times are fairer than two AIMD flows (Jain 0.84 against 0.64 at +150 ms) but still favour the short-RTT flow.
4. **Interpretability and verification** [12]. A depth-5 decision tree agrees with the network on 79% of decisions yet matches its transfer time on four of five paths; noise of 0.2 on the inputs changes the time by at most 12%; formal certification works only at tiny perturbations.
5. **Coexistence with kernel TCP.** `netem_coexist.sh` runs a kernel TCP flow and one RUDP transfer on the same bottleneck: the neural agent took 24% to 34% of the link and tuned AIMD 5% to 17%, so it does not starve TCP (3 runs per cell, one laptop).

Key researcher: Van Jacobson

Van Jacobson, then at Lawrence Berkeley Laboratory, is the lead author of "Congestion Avoidance and Control" [1], the paper behind TCP slow start, congestion avoidance and the retransmit-timer fixes. RFC 5681 states that its four congestion-control algorithms were developed in his work [3]. Every congestion-control mechanism in RUDP traces back to that paper, which is why we name him as the researcher to follow in this area.

Bibliography

1. V. Jacobson and M. J. Karels, "Congestion Avoidance and Control," Proc. ACM SIGCOMM '88, pp. 314-329, 1988.
2. D.-M. Chiu and R. Jain, "Analysis of the Increase and Decrease Algorithms for Congestion Avoidance in Computer Networks," Computer Networks and ISDN Systems, vol. 17, no. 1, pp. 1-14, 1989.
3. M. Allman, V. Paxson and E. Blanton, "TCP Congestion Control," RFC 5681, IETF, Sept. 2009.
4. S. Ha, I. Rhee and L. Xu, "CUBIC: A New TCP-Friendly High-Speed TCP Variant," ACM SIGOPS Operating Systems Review, vol. 42, no. 5, pp. 64-74, July 2008.
5. A. Langley et al., "The QUIC Transport Protocol: Design and Internet-Scale Deployment," Proc. ACM SIGCOMM '17, Los Angeles, CA, Aug. 2017.
6. J. Iyengar and M. Thomson, Eds., "QUIC: A UDP-Based Multiplexed and Secure Transport," RFC 9000, IETF, May 2021.
7. S. Hemminger, "Network Emulation with NetEm," Open Source Development Lab, Apr. 2005, as cited in the tc-netem manual.
8. K. S. Midhula and P. Arun Raj Kumar, "An Adaptive Congestion Control Protocol for Wireless Networks Using Deep Reinforcement Learning," IEEE Transactions on Network and Service Management, vol. 21, no. 2, pp. 2027-2043, 2024. doi: 10.1109/TNSM.2023.3325543.
9. Y. Cao, J. Nie, Y. Fan, X. Shao and G. Lei, "DRLFcc: Deep Reinforcement Learning-empowered Congestion Control Mechanism for TCP Fast Recovery in High Loss Wireless Networks," in Proc. IEEE GLOBECOM 2023, pp. 4345-4350, 2023. doi: 10.1109/GLOBECOM54140.2023.10437070.
10. A. Mazilu, B. Giacomoni and I. Parisis, "Learning-Based vs Human-Derived Congestion Control: An In-Depth Experimental Study," arXiv:2510.25105, 2025.
11. Tian et al., "Fairness generalizability of learned congestion control" (Jury), Proc. ACM EuroSys 2025. Title and venue as found in the search; details not checked.
12. Yang et al., "C3: Learning Congestion Controllers with Formal Certificates," arXiv:2412.10915, 2024.
13. N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh and V. Jacobson, "BBR: Congestion-Based Congestion Control," ACM Queue, vol. 14, no. 5, 2016.

References 1 to 7 were checked against publisher, repository or RFC Editor pages on 8 October 2026. References 8 and 9 are taken from the project abstract and were not re-checked. References 10 and 12 were read as abstract pages on arXiv; reference 11 is known only from a search listing; reference 13 is cited from memory as a well-known paper. All four should be verified by the authors.